

SET interface implementing classes

HashSet, LinkedHashSet and TreeSet are Java Set implementations that store unique elements but differ in ordering and performance. HashSet is fastest (unordered, $O(1)$), LinkedHashSet maintains insertion order ($O(1)$), and TreeSet keeps elements sorted (natural or Comparator, $O(\log n)$). HashSet is best for speed, LinkedHashSet for order and TreeSet for sorting.

HASHSET:

Structure: backed by HashMap.

Order: No guarantee of order, elements can change over time.

Performance: Fastest for insertion, removal and lookup, providing $O(1)$ average time complexity.

Nulls: allows one null element.

Use case: when unique set of elements is needed but order is not important.

LINKEDHASHSET:

Structure: Hashtable + LinkedList.

Order: maintains the insertion order.

Performance: slightly slower than HashSet due to maintaining the LinkedList, still $O(1)$.

Nulls: allows one null element.

Use case: when you need to maintain insertion order, such as when iterating through items in the order they were added.

TREESET:

Structure: Red-Black tree (navigable map/TreeMap)

Order: maintains elements in sorted (natural) order or by a specified Comparator.

Performance: slower, offering $O(\log n)$ time complexity for basic operations.

Nulls: does not allow null, throws NullPointerException.

Use case: when you need to keep elements sorted or perform range-based queries.

	HASHSET	LINKEDHASHSET	TREESET
ORDERING	Unordered	Insertion order	Sorted order
PERFORMANCE	Best $O(1)$	Good $O(1)$	Slower $O(\log n)$
IMPLEMENTATION	HashMap	LinkedHashMap	TreeMap
NULL ELEMENTS	Allowed	Allowed	Not allowed
DATA STRUCTURE	Hashtable	Hashtable + LinkedList	Red-Black tree

USE CASES:

Default choice: HashSet for maximum performance.

Need order: LinkedHashSet if insertion order matters.

Need sorted: TreeSet if sorting is required.

MAP interface implementing classes

HashMap, LinkedHashMap, Hashtable and TreeMap are all Java Map implementations, but they differ in ordering, performance and synchronization. HashMap is fast and unsorted, LinkedHashMap maintains insertion order, TreeMap sorts keys and Hashtable is a synchronized, legacy alternative to HashMap.

ORDER:

HashMap: No guarantee of order, order can change over time.

LinkedHashMap: maintains insertion order.

TreeMap: sorts in natural ascending order or a custom Comparator.

Hashtable: No guaranteed order.

PERFORMANCE:

HashMap/LinkedHashMap/Hashtable: $O(1)$ constant time for basic operations (get/put).

TreeMap: $O(\log n)$ logarithmic time for basic operations.

NULL KEYS/VALUES:

HashMap/LinkedHashMap: allows one null key and multiple null values.

TreeMap: does not allow null key but allows multiple null values.

Hashtable: does not allow null as both keys and values.

SYNCHRONIZATION:

HashMap/LinkedHashMap/TreeMap: not synchronized (not thread safe).

Hashtable: synchronized (thread safe), which makes it slower.

USE CASES:

HashMap: default choice for key-value storage when order does not matter.

LinkedHashMap: when you need to maintain insertion order for predictable iteration.

TreeMap: when you need keys sorted implicitly or require range-based operations.

Hashtable: avoid in new code, use ConcurrentHashMap for thread-safe maps.

	HASHMAP	LINKEDHASHMAP	TREEMAP	HASHTABLE
ORDERING	Unordered	Insertion order	Sorted order	Unordered
PERFORMANCE	Fast $O(1)$	Slightly slow $O(1)$	Slower $O(\log n)$	Slower to HashMap
IMPLEMENTATION	Hashtable	Hashtable+linkedlist	Red Black tree	Hashtable
NULL KEYS	Only one	Only one	Not allowed	Not allowed
NULL VALUES	Multiple	Multiple	Multiple	Not allowed
THREAD SAFETY	Not Thread safe	Not Thread safe	Not Thread safe	Synchronized Thread safe
INTRODUCED IN	Java 1.2	Java 1.4	Java 1.2	Java 1.0 (legacy)